Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

Department of Information Technology

ASSIGNMENT 2

NAME	Viraj Shrikant Pathare
PRN	124B1F149
CLASS	SY IT
DIVISION	A

#Command to display existing databases.

```
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.00 sec)
```

#Command to create the Employee Appraisal System database.

```
mysql> CREATE DATABASE appraisal_system;
Query OK, 1 row affected (0.01 sec)

mysql> USE appraisal_system;
Database changed
```

Part a: SQL DDL Statements

#Create Department table.

```
mysql> CREATE TABLE Department (
-> dept_id INT PRIMARY KEY AUTO_INCREMENT,
-> dept_name VARCHAR(100) NOT NULL UNIQUE,
-> location VARCHAR(100)
-> );
Query OK, 0 rows affected (0.05 sec)
```

#Create Employee table with constraints.

```
mysql> CREATE TABLE Employee (
-> emp_id INT PRIMARY KEY AUTO_INCREMENT,
-> name VARCHAR(100) NOT NULL,
-> email VARCHAR(150) NOT NULL UNIQUE,
-> phone VARCHAR(15),
-> dept_id INT,
```

```
-> designation VARCHAR(80),
-> joining_date DATE NOT NULL DEFAULT (CURDATE()),
-> salary DECIMAL(10,2) NOT NULL CHECK (salary > 0),
-> is_active TINYINT DEFAULT 1,
-> FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
-> );
Query OK, 0 rows affected (0.08 sec)
```

#Create Appraisal table.

```
mysql> CREATE TABLE Appraisal (
-> appraisal_id INT PRIMARY KEY AUTO_INCREMENT,
-> emp_id INT NOT NULL,
-> reviewer_id INT NOT NULL,
-> appraisal_date DATE NOT NULL,
-> rating DECIMAL(3,1) CHECK (rating BETWEEN 1.0 AND 5.0),
-> comments TEXT,
-> status ENUM('Pending','Completed','Approved') DEFAULT 'Pending',
-> FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),
-> FOREIGN KEY (reviewer_id) REFERENCES Employee(emp_id)
-> );
Query OK, 0 rows affected (0.07 sec)
```

#Create Goals table.

```
mysql> CREATE TABLE Goals (
-> goal_id INT PRIMARY KEY AUTO_INCREMENT,
-> emp_id INT NOT NULL,
-> description VARCHAR(255) NOT NULL,
-> target_date DATE,
-> achieved TINYINT DEFAULT 0,
-> FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)
-> );
Query OK, 0 rows affected (0.05 sec)
```

#Create Training table.

```
mysql> CREATE TABLE Training (
-> training_id INT PRIMARY KEY AUTO_INCREMENT,
-> emp_id INT NOT NULL,
-> training_name VARCHAR(150) NOT NULL,
-> completion_date DATE,
-> score DECIMAL(5,2) CHECK (score BETWEEN 0 AND 100),
-> FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)
-> );
Query OK, 0 rows affected (0.05 sec)
```

#Create Salary_History table.

```
mysql> CREATE TABLE Salary_History (
-> history_id INT PRIMARY KEY AUTO_INCREMENT,
-> emp_id INT NOT NULL,
-> old_salary DECIMAL(10,2),
-> new_salary DECIMAL(10,2),
-> effective_date DATE NOT NULL,
-> reason VARCHAR(200),
-> FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)
-> );
Query OK, 0 rows affected (0.06 sec)
```

#Create Index on Employee email and salary.

```
mysql> CREATE INDEX idx_emp_email ON Employee(email);
Query OK, 0 rows affected (0.03 sec)
mysql> CREATE INDEX idx_emp_salary ON Employee(salary);
Query OK, 0 rows affected (0.02 sec)
mysql> CREATE INDEX idx_appr_emp ON Appraisal(emp_id, appraisal_date);
Query OK, 0 rows affected (0.02 sec)
```

#Display all tables.

```
mysql> SHOW TABLES;
+                    +
| Tables_in_appraisal_system |
+                    +
| Appraisal |
| Department |
| Employee |
| Goals |
| Salary_History |
| Training |
+                    +
6 rows in set (0.00 sec)
```

#Describe Employee table.

```
mysql> DESC Employee;
+--------------+---------------+------+-----+---------+ +
| Field | Type | Null | Key | Default | Extra |
+--------------+---------------+------+-----+---------+ +
| emp_id | int | NO | PRI | NULL | auto_increment |
| name | varchar(100) | NO | | NULL | |
| email | varchar(150) | NO | UNI | NULL | |
| phone | varchar(15) | YES | | NULL | |
| dept_id | int | YES | MUL | NULL | |
| designation | varchar(80) | YES | | NULL | |
| joining_date | date | NO | | NULL | |
| salary | decimal(10,2) | NO | | NULL | |
| is_active | tinyint(1) | YES | | 1 | |
+--------------+---------------+------+-----+---------+ +
9 rows in set (0.00 sec)
```

Part b: SQL DML Statements

#Insert data into Department.

```
mysql> INSERT INTO Department (dept_name, location) VALUES
-> ('Engineering', 'Pune'),
-> ('Human Resources','Mumbai'),
-> ('Finance', 'Pune'),
-> ('Marketing', 'Delhi'),
-> ('IT Operations', 'Bangalore');
Query OK, 5 rows affected (0.01 sec)
```

#Insert data into Employee.

```
mysql> INSERT INTO Employee (name,email,phone,dept_id,designation,joining_date,salary) VALUES
-> ('Rahul Patil', 'rahul@company.com', '9876543210', 1, 'Software Engineer','2022-01-10', 65000),
-> ('Sneha Sharma', 'sneha@company.com', '9823456780', 1, 'Senior Engineer', '2020-03-15', 90000),
-> ('Amit Verma', 'amit@company.com', '9512345678', 2, 'HR Manager', '2019-07-01', 75000),
-> ('Pooja Nair', 'pooja@company.com', '9234567890', 3, 'Finance Analyst', '2021-05-20', 60000),
-> ('Karan Mehta', 'karan@company.com', '9345678901', 4, 'Marketing Lead', '2023-02-01', 55000);
Query OK, 5 rows affected (0.01 sec)
```

#Insert data into Appraisal.

```
mysql> INSERT INTO Appraisal (emp_id,reviewer_id,appraisal_date,rating,comments,status) VALUES
-> (1, 2, '2024-12-31', 4.2, 'Excellent performance, exceeded targets.', 'Completed'),
-> (2, 3, '2024-12-31', 4.8, 'Outstanding leadership and delivery.', 'Approved'),
-> (3, 2, '2024-12-31', 3.5, 'Good work, needs improvement in hiring.', 'Completed'),
-> (4, 3, '2024-12-31', 4.0, 'Strong analytical skills demonstrated.', 'Completed'),
-> (5, 2, '2024-12-31', 3.8, 'Good campaign execution.', 'Pending');
Query OK, 5 rows affected (0.01 sec)
```

#Insert data into Goals.

```
mysql> INSERT INTO Goals (emp_id, description, target_date, achieved) VALUES
-> (1, 'Complete React certification', '2024-06-30', 1),
-> (1, 'Reduce bug backlog by 30%', '2024-12-31', 1),
-> (2, 'Mentor 2 junior engineers', '2024-12-31', 1),
-> (3, 'Reduce hiring cycle to 30 days', '2024-09-30', 0),
-> (4, 'Automate monthly financial reports','2024-12-31', 1);
Query OK, 5 rows affected (0.01 sec)
```

#Insert data into Training.

```
mysql> INSERT INTO Training (emp_id, training_name, completion_date, score) VALUES
-> (1, 'Advanced Java', '2024-03-15', 88.50),
-> (1, 'AWS Cloud Practitioner','2024-07-20', 92.00),
-> (2, 'Leadership Workshop', '2024-04-10', 95.00),
-> (3, 'HR Analytics', '2024-05-25', 80.00),
-> (4, 'Excel Mastery', '2024-06-15', 85.00);
Query OK, 5 rows affected (0.01 sec)
```

DML Query 1: SELECT all employees.

```
mysql> SELECT * FROM Employee;
+--------+--------------+--------------------+------------+---------+-------------------+----
---
-------+-----------+  +
| emp_id | name | email | phone | dept_id | designation | joining_date | salary | is_active |
+--------+--------------+--------------------+------------+---------+-------------------+----
---
-------+-----------+  +
| 1 | Rahul Patil | rahul@company.com | 9876543210 | 1 | Software Engineer | 2022-01-10 |
65000.00 | 1 |
| 2 | Sneha Sharma | sneha@company.com | 9823456780 | 1 | Senior Engineer | 2020-03-15 |
90000.00
| 1 |
| 3 | Amit Verma | amit@company.com | 9512345678 | 2 | HR Manager | 2019-07-01 | 75000.00 | 1
|
| 4 | Pooja Nair | pooja@company.com | 9234567890 | 3 | Finance Analyst | 2021-05-20 |
60000.00 |
1 |
| 5 | Karan Mehta | karan@company.com | 9345678901 | 4 | Marketing Lead | 2023-02-01 |
55000.00 |
1 |
+--------+--------------+--------------------+------------+---------+-------------------+----
---
-------+-----------+  +
5 rows in set (0.00 sec)
```

DML Query 2: SELECT employees with salary > 60000.

```
mysql> SELECT name, designation, salary FROM Employee WHERE salary > 60000;
+--------------+-------------------+-+
| name | designation | salary |
+--------------+-------------------+-+
| Sneha Sharma | Senior Engineer | 90000.00 |
| Amit Verma | HR Manager | 75000.00 |
| Rahul Patil | Software Engineer | 65000.00 |
+--------------+-------------------+-+
3 rows in set (0.00 sec)
```

DML Query 3: UPDATE salary of Karan Mehta.

```
mysql> UPDATE Employee SET salary = 60000 WHERE name = 'Karan Mehta';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

DML Query 4: SELECT using BETWEEN operator.

```
mysql> SELECT name, salary FROM Employee WHERE salary BETWEEN 60000 AND 80000;
+-------------+       +
| name | salary |
+-------------+       +
| Rahul Patil | 65000.00 |
| Amit Verma | 75000.00 |
| Pooja Nair | 60000.00 |
| Karan Mehta | 60000.00 |
+-------------+       +
4 rows in set (0.00 sec)
```

DML Query 5: SELECT using LIKE operator.

```
mysql> SELECT name, email FROM Employee WHERE email LIKE '%company.com';
+--------------+      +
| name | email |
+--------------+      +
| Rahul Patil | rahul@company.com |
| Sneha Sharma | sneha@company.com |
| Amit Verma | amit@company.com |
| Pooja Nair | pooja@company.com |
| Karan Mehta | karan@company.com |
+--------------+      +
5 rows in set (0.00 sec)
```

DML Query 6: Aggregate – COUNT and AVG salary by department.

```
mysql> SELECT dept_id, COUNT(*) AS total_emp, AVG(salary) AS avg_salary
-> FROM Employee GROUP BY dept_id;
+---------+-----------+      +
| dept_id | total_emp | avg_salary |
+---------+-----------+      +
| 1 | 2 | 77500.0000 |
| 2 | 1 | 75000.0000 |
| 3 | 1 | 60000.0000 |
| 4 | 1 | 60000.0000 |
+---------+-----------+      +
4 rows in set (0.00 sec)
```

DML Query 7: SELECT with ORDER BY (salary descending).

```
mysql> SELECT name, salary FROM Employee ORDER BY salary DESC;
+--------------+      +
| name | salary |
+--------------+      +
| Sneha Sharma | 90000.00 |
| Amit Verma | 75000.00 |
| Rahul Patil | 65000.00 |
| Pooja Nair | 60000.00 |
```

```
5 rows in set (0.00 sec)
```

DML Query 8: SELECT using IN operator (multiple departments).

```
mysql> SELECT name, dept_id, designation FROM Employee WHERE dept_id IN (1, 3);
+--------------+---------+-----------------+
| name | dept_id | designation |
+--------------+---------+-----------------+
| Rahul Patil | 1 | Software Engineer |
| Sneha Sharma | 1 | Senior Engineer |
| Pooja Nair | 3 | Finance Analyst |
+--------------+---------+-----------------+
3 rows in set (0.00 sec)
```

DML Query 9: SELECT using string function UPPER().

```
mysql> SELECT UPPER(name) AS emp_name, email FROM Employee;
+--------------+-----------------+
| emp_name | email |
+--------------+-----------------+
| RAHUL PATIL | rahul@company.com |
| SNEHA SHARMA | sneha@company.com |
| AMIT VERMA | amit@company.com |
| POOJA NAIR | pooja@company.com |
| KARAN MEHTA | karan@company.com |
+--------------+-----------------+
5 rows in set (0.00 sec)
```

DML Query 10: DELETE an employee record (using test record).

```
mysql> INSERT INTO Employee (name,email,phone,dept_id,designation,joining_date,salary)
-> VALUES ('Test User','test@test.com','0000000000',1,'Intern','2024-01-01',20000);
Query OK, 1 row affected (0.00 sec)

mysql> DELETE FROM Employee WHERE email = 'test@test.com';
Query OK, 1 row affected (0.01 sec)
```

DML Query 11: SET operator – UNION of engineering and HR employee names.

```
mysql> SELECT name FROM Employee WHERE dept_id = 1
-> UNION
-> SELECT name FROM Employee WHERE dept_id = 2;
+--------------+
| name |
+--------------+
| Rahul Patil |
| Sneha Sharma |
| Amit Verma |
+--------------+
3 rows in set (0.00 sec)
```

Theory

Q1. What is DDL and which DDL commands did you use?

DDL (Data Definition Language) defines and manages database structure. Commands used: CREATE (database, tables), ALTER (add/drop columns), DROP, CREATE INDEX, DESC.

Q2. What is DML and which operations did you perform?

DML (Data Manipulation Language) manages data inside tables. Operations performed: INSERT (add records), SELECT (retrieve data with conditions, functions), UPDATE (modify records), DELETE (remove records), UNION (set operation).

Q3. What is a Primary Key? Give an example from your project.

A Primary Key uniquely identifies each record in a table and cannot be NULL or duplicate. Example: emp_id is the primary key of the Employee table.

Q4. What is a Foreign Key? Give an example from your project.

A Foreign Key is a column that references the Primary Key of another table, establishing referential integrity. Example: dept_id in Employee references dept_id in Department.

Q5. What is the purpose of constraints in SQL?

Constraints enforce data integrity rules: NOT NULL prevents null values, UNIQUE ensures no duplicates, CHECK validates range/format, DEFAULT sets a fallback value, and FOREIGN KEY maintains referential integrity between tables.